ESP32 WiFi Configuration Manager

A beautiful, beginner-friendly WiFi configuration system for ESP32 with captive portal, EEPROM storage, and a modern web interface.

✨ Features

- 🎯 **Captive Portal** - Automatic redirect when connecting (like coffee shop WiFi)
- 📡 **WiFi Scanning** - Discover and display all available networks
- 🔐 **Secure Storage** - Save credentials in ESP32's persistent memory (Preferences)
- 🌐 **DHCP or Static IP** - Choose automatic or manual IP configuration
- 🎨 **Beautiful Interface** - Modern, responsive web design
- 🔄 **Auto-Reconnect** - Remembers settings and reconnects on power-up
- ⚡ **Disconnect Handling** - Automatic reconnection with configurable retry logic
- ⏱️ **Connection Timeout** - Prevents infinite waiting during connection attempts
- 📱 **Mobile Friendly** - Works on phones, tablets, and computers

📦 Required Libraries

All libraries are built into ESP32 Arduino Core - no additional installation needed!

- WiFi (built-in)
- WebServer (built-in)
- DNSServer (built-in)
- Preferences (built-in)

🚀 Quick Start Guide

1. Install Arduino IDE & ESP32 Support

1. Download Arduino IDE
2. Open Arduino IDE → File → Preferences
3. Add to "Additional Board Manager URLs":

```
https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json
```

4. Go to Tools → Board → Boards Manager
5. Search for "ESP32" and install "esp32 by Espressif Systems"

2. Upload the Code

1. Open `ESP32_WiFi_Manager.ino` in Arduino IDE
2. Select your board: Tools → Board → ESP32 Arduino → (your ESP32 board)
3. Select your port: Tools → Port → (your COM port)
4. Click the Upload button (→)

3. Configure Your WiFi

First Time Setup:

1. Power on your ESP32
2. On your phone/computer, open WiFi settings
3. Connect to the network named **"ESP32-Setup"** (no password needed)
4. A configuration page should open automatically
   - If not, open a browser and go to: `http://192.168.4.1`
5. On the configuration page:
   - Click "Scan for Networks"
   - Select your WiFi network from the list
   - Enter your WiFi password
   - Choose DHCP (automatic) or Static IP
   - Click "Connect to Network"
6. The ESP32 will connect and display its IP address
7. That's it! The ESP32 will remember these settings forever

🎮 How It Works

Configuration Mode (First Time)

```
ESP32 starts → No saved WiFi → Creates "ESP32-Setup" AP
↓
You connect to "ESP32-Setup"
```

```
↓
Captive portal opens automatically
↓
Select network, enter password, click connect
↓
Settings saved to EEPROM → ESP32 restarts
```

Normal Operation (After Configuration)

```
ESP32 starts → Loads saved settings → Connects to your WiFi
↓
Connected! IP address displayed in Serial Monitor
↓
Your application code runs
```

🔧 Configuration Options

DHCP (Automatic IP) - Recommended for Beginners

- ✅ Easiest option
- ✅ No network knowledge required
- ✅ Works on most networks
- Router assigns IP automatically

Static IP (Manual Configuration)

- Requires entering:
  - IP Address (e.g., 192.168.1.100)
  - Gateway (usually 192.168.1.1)
  - Subnet Mask (usually 255.255.255.0)
  - DNS Servers (e.g., 8.8.8.8)

📊 Serial Monitor Output

Open Serial Monitor (Tools → Serial Monitor) at 115200 baud to see:

```
=== ESP32 WiFi Manager ===
Found saved WiFi credentials
SSID: YourNetwork

=== Connecting to WiFi ===
Using DHCP
.....
✓ WiFi Connected!
IP Address: 192.168.1.50
Gateway: 192.168.1.1
Subnet: 255.255.255.0
DNS: 8.8.8.8
```

🎨 Web Interface Features

Network Scanner

- Lists all available WiFi networks
- Shows signal strength (bars + dBm)
- Indicates secure (🔒) vs open (🔓) networks
- Click to select a network

Smart Forms

- Auto-fills selected network name
- Password field for security
- Toggle between DHCP and Static IP
- Validates IP addresses

Status Messages

- Real-time connection feedback
- Success/error notifications
- Shows assigned IP address after connection

🔄 Resetting Configuration

To reset and reconfigure:

Method 1 - Clear from code:

```cpp
void setup() {
  preferences.begin("wifi-config", false);
  preferences.clear();  // Add this line
  preferences.end();
  // Rest of setup code...
}
```

Method 2 - Upload blank sketch: Upload the code with an empty SSID, which will trigger config mode.

🛠️ Customization

Change Access Point Name

```cpp
#define AP_SSID "ESP32-Setup"  // Change this
```

Add Password to AP

```cpp
#define AP_PASSWORD "your-password"  // Empty = no password
```

Change Timeout

```cpp
#define CONFIG_TIMEOUT 300000  // 5 minutes in milliseconds
```

Configure Reconnection Behavior

```cpp
#define WIFI_CONNECT_TIMEOUT 20000       // Connection timeout (20 seconds)
#define WIFI_RECONNECT_INTERVAL 30000    // Time between reconnection attempts
#define MAX_RECONNECT_ATTEMPTS 5         // Max attempts before config mode
```

🔌 Disconnect Handling

The system includes robust disconnect handling:

Automatic Reconnection

- **Detection**: WiFi events monitor connection status in real-time
- **Retry Logic**: Automatically attempts reconnection every 30 seconds
- **Max Attempts**: After 5 failed attempts, enters configuration mode
- **Timeout**: Each connection attempt has a 20-second timeout

Connection Monitoring

The ESP32 checks WiFi status every 5 seconds and logs:

- Connection status changes
- Signal strength warnings (below -80 dBm)
- Disconnection reasons
- Reconnection attempts

Serial Monitor Output (Disconnect Example)

```
✓ WiFi connected
IP address: 192.168.1.50

✗ WiFi disconnected!

=== Handling Disconnect ===
Disconnect reason: 6

=== Reconnection Attempt #1 ===
Attempting to reconnect to: YourNetwork
.....
✓ Reconnection successful!
```

What Happens During Disconnect

1. **Immediate Detection -** WiFi event triggers disconnect handler
2. **Wait Period -** System waits 30 seconds before first retry
3. **Reconnection Attempts -** Up to 5 attempts with timeout protection
4. **Fallback -** If all attempts fail, enters configuration mode
5. **User Notification -** Can reconfigure WiFi credentials

Customizing Disconnect Behavior

Aggressive Reconnection (faster, more battery usage):

```cpp
#define WIFI_RECONNECT_INTERVAL 10000  // Try every 10 seconds
```

Conservative Reconnection (slower, saves battery):

```cpp
#define WIFI_RECONNECT_INTERVAL 60000  // Try every minute
```

More Patience Before Config Mode:

```cpp
#define MAX_RECONNECT_ATTEMPTS 10  // 10 attempts instead of 5
```

Clear Credentials on Repeated Failure (uncomment in code):

```cpp
// In attemptReconnect() function:
if (reconnectAttempts >= MAX_RECONNECT_ATTEMPTS) {
  preferences.clear();  // Clear saved credentials
  startConfigMode();
}
```

Change Timeout

```cpp
#define CONFIG_TIMEOUT 300000  // 5 minutes in milliseconds
```

Modify Colors/Style

The web interface uses CSS variables. Edit these in the `<style>` section:

```css
```

```
:root {
  --primary: #00d4ff;      /* Main accent color */
  --bg-dark: #0a0e1a;      /* Background */
  --text: #e8eaed;         /* Text color */
  /* ... more variables ... */
}
```

💡 Using with Your Application

After WiFi is connected, add your code in the `loop()` function:

```cpp
void loop() {
  if (configMode) {
    // Handle configuration mode
    dnsServer.processNextRequest();
    server.handleClient();
  }

  // Your application code here
  if (!configMode && WiFi.status() == WL_CONNECTED) {
    // Your web server, sensors, etc.
    // Example:
    // handleYourWebServer();
    // readSensors();
    // sendData();
  }

  delay(10);
}
```

🐛 Troubleshooting

Captive Portal Doesn't Open

- Try manually navigating to `http://192.168.4.1`
- Some devices disable captive portals - use browser instead
- Try forgetting and reconnecting to "ESP32-Setup"

Can't Connect to Saved Network

- ESP32 will automatically enter config mode if connection fails
- Check your WiFi password is correct
- Ensure 2.4GHz network (ESP32 doesn't support 5GHz)

ESP32 Keeps Restarting

- Check Serial Monitor for error messages
- Verify power supply is adequate (500mA minimum)
- Try uploading code again

ESP32 Keeps Restarting

- Check Serial Monitor for error messages
- Verify power supply is adequate (500mA minimum)
- Try uploading code again

Frequent Disconnections

- Check WiFi signal strength (should be above -80 dBm)
- Move ESP32 closer to router
- Check for interference from other devices
- Try changing WiFi channel on your router
- Consider using Static IP instead of DHCP

ESP32 Won't Reconnect After Disconnect

- Check Serial Monitor for reconnection attempts
- Verify router is still broadcasting the network
- Try power cycling the ESP32
- If max attempts reached, ESP32 enters config mode automatically

Static IP Not Working

- Verify all IP addresses are in same subnet
- Check gateway is correct (usually your router's IP)
- Try DHCP mode first to verify network connectivity

📝 Technical Details

Memory Usage

- Preferences library replaces EEPROM (more reliable)
- Configuration survives power cycles and resets
- Stores: SSID, password, IP settings

Network Modes

- **AP Mode**: Creates access point for configuration
- **STA Mode**: Connects to your WiFi network

DNS Server

- Redirects all DNS requests to ESP32
- Creates captive portal effect
- Works with most devices (iOS, Android, Windows, Mac)

🔒 Security Notes

- WiFi password stored in ESP32 flash memory
- Access Point is open by default (add password if needed)
- Configuration mode times out after 5 minutes
- Use HTTPS in production applications

🎯 Best Practices

1. **Test DHCP First**: Always try automatic IP before static
2. **Use Serial Monitor**: Helps debug connection issues
3. **Add Timeout**: Don't let config mode run forever
4. **Save Power**: Put ESP32 in sleep mode when not active
5. **Handle Disconnections**: Add reconnection logic for robustness

📚 Additional Resources

- ESP32 Documentation

- Arduino ESP32 Core
- WiFi Library Reference

🤝 Contributing

Feel free to improve this code! Some ideas:

- Add WPS support
- Implement WiFi strength meter
- Add multiple AP credentials
- Create mobile app for configuration
- Add OTA (Over-The-Air) updates

📄 License

This code is open source and free to use in your projects!

Made with ❤️ for makers and beginners

Questions? Check the Serial Monitor output - it's very helpful for debugging!